项目介绍

这是一个基于大语言模型的智能面试辅助平台，旨在帮助求职者提升面试能力。系统提供三大核心功能：

1. **智能简历分析**：上传简历后，AI 自动进行多维度评分并给出改进建议
2. **模拟面试系统**：基于简历内容生成个性化面试题，支持实时问答和答案评估
3. **RAG 知识库问答**：上传技术文档构建私有知识库，支持向量检索增强的智能问答

开源地址（欢迎 Star 鼓励）：

- Github：https://github.com/Snailclimb/interview-guide
- Gitee：https://gitee.com/SnailClimb/interview-guide

配套教程内容安排

本项目专为求职者和开发者设计，旨在解决当前 AI 应用开发中的核心痛点。通过“保姆级”的保姆级教程，我们将从零构建一个融合了 **大模型集成、RAG（检索增强生成）、高性能对象存储与向量数据库** 的完整系统。

无论你是想学习 **Spring AI** 的前沿应用，还是需要一个**高含金量的简历项目**，本项目都将为你提供从基建搭建、业务攻坚到面试话术复盘的全方位指导。

内容安排如下（正在持续更新中）：

环境搭建

- [x] 本地搭建 PostgreSQL + PGvector 向量数据库
- [x] Spring Boot + RustFS 构建高性能 S3 兼容的对象存储服务
- [x] ☆大模型 API 申请和 Ollama 部署本地模型
- [x] 环境搭建终章与项目启动

核心功能开发

- [x] 基于 Tika 实现多格式内容提取与解析
- [x] ☆Spring AI 与大模型集成
- [x] ☆Spring AI + pgvector 实现 RAG 知识库问答
- [x] 基于 SSE 实现打字机效果输出
- [x] 手把手教你写出生产级结构化 Prompt
- [x] AI 模拟面试功能
- [x] 基于 iText 8 实现 PDF 报告导出

进阶优化

- [x] MapStruct 实体映射最佳实践
- [x] ☆基于 Redis Stream 的异步任务处理实现
- [x] 封装 Redis + Lua 多维度分布式限流组件
- [x] Spring Boot 4.0 升级指南

- [ ] Docker Compose 一键部署

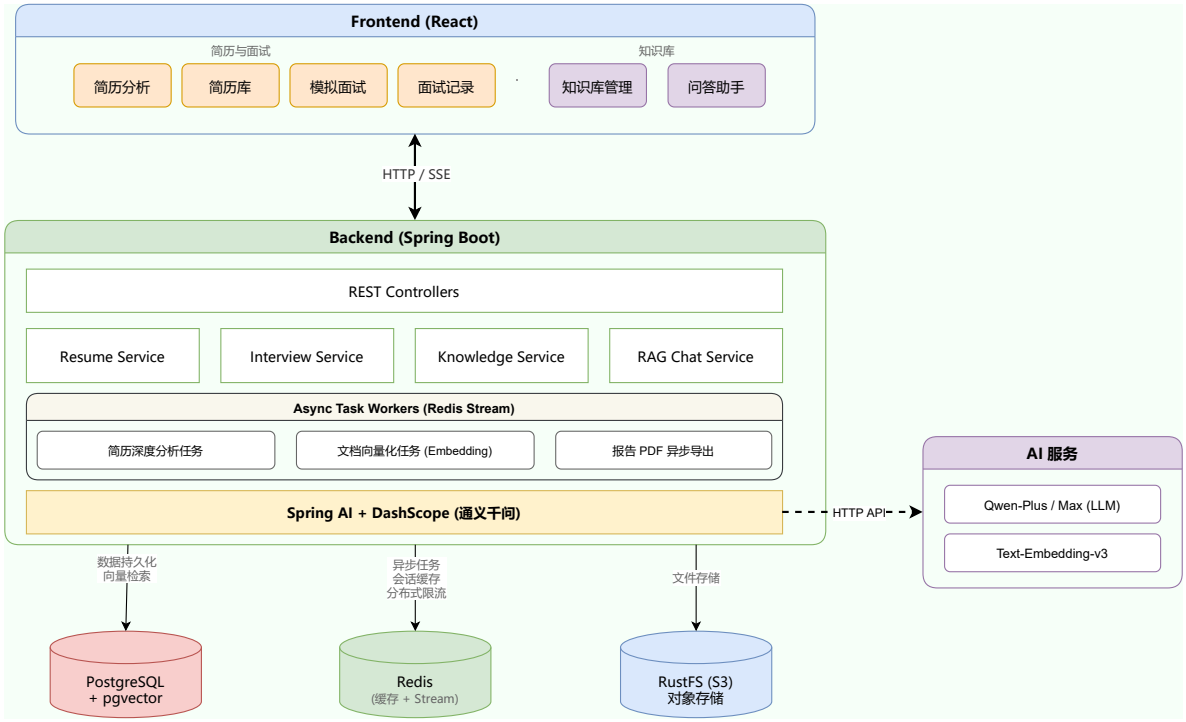
面试

- [x] ☆简历编写与项目经历深度包装指南
- [x] 面试官问“这个项目哪里来的”时，如何回答？
- [x] ☆Spring AI 面试问题挖掘
- [x] ☆知识库 RAG 面试问题挖掘
- [x] Redis 面试问题挖掘
- [x] 文件上传解析与 PDF 导出面试问题挖掘

系统架构

提示：架构图采用 draw.io 绘制，导出为 svg 格式，在 Dark 模式下的显示效果会有问题。

系统采用前后端分离架构，整体分为三层：前端展示层、后端服务层、数据存储层。



后端层：

- REST Controllers：统一的 API 入口，处理 HTTP 请求
- 业务服务层：
  - Resume Service：简历上传、解析、AI 分析
    - Interview Service：面试会话管理、问题生成、答案评估
    - Knowledge Service：知识库上传、文本分块、向量化
    - RAG Chat Service：检索增强生成，流式问答
- 异步处理层：基于 Redis Stream 的消费者，异步处理耗时的 AI 任务（如简历分析、向量化、面试评估）
- AI 集成层：Spring AI + DashScope（通义千问）。统一的 LLM 调用接口，支持对话生成和文本向量化。

数据存储层：

- PostgreSQL + pgvector：

    - 关系数据：简历、面试记录、知识库元数据
    - 向量检索：存储文档向量，支持相似度搜索
- Redis：
    - 会话缓存：面试会话状态
    - 消息队列：Redis Stream 实现异步任务队列
- RustFS/MinIO (S3)：原始文件（简历 PDF、知识库文档）

异步处理流程：

简历分析、知识库向量化和面试报告生成采用 Redis Stream 异步处理，这里以简历分析和知识库向量化为例介绍一下整体流程：

```
上传请求 → 保存文件 → 发送消息到 Stream → 立即返回
                                ↓
                        Consumer 消费消息
                                ↓
                        执行分析/向量化任务
                                ↓
                        更新数据库状态
                                ↓
                        前端轮询获取最新状态
```

状态流转：`PENDING` → `PROCESSING` → `COMPLETED` / `FAILED`

知识库问答处理流程：

```
知识库问答 → 问题向量化 → pgvector 相似度搜索 → 检索相关文档
                                                    ↓
                                        构建 Prompt → LLM 生成回答 → SSE 流式返回
```

技术栈概览

后端技术

技术	版本	说明
Spring Boot	4.0	应用框架
Java	21	开发语言
Spring AI	2.0	AI 集成框架
PostgreSQL + pgvector	14+	关系数据库 + 向量存储
Redis	6+	缓存 + 消息队列（Stream）
Apache Tika	2.9.2	文档解析
iText 8	8.0.5	PDF 导出
MapStruct	1.6.3	对象映射
Gradle	8.14	构建工具

前端技术

技术	版本	说明
React	18.3	UI 框架
TypeScript	5.6	开发语言
Vite	5.4	构建工具
Tailwind CSS	4.1	样式框架
React Router	7.11	路由管理
Framer Motion	12.23	动画库
Recharts	3.6	图表库
Lucide React	0.468	图标库

技术选型常见问题解答

这里只是简单介绍，后续我会分享文章详细拷打技术选型。

为什么选择 Spring AI?

Spring AI 是 Spring 官方推出的 AI 集成框架，提供了统一的 LLM 调用抽象。选择它的原因：

1. 统一抽象：一套代码支持多种 LLM 提供商（OpenAI、阿里云 DashScope、Ollama 等），切换模型只需修改配置
2. Spring 生态集成：与 Spring Boot 无缝集成，支持自动配置、依赖注入、声明式调用
3. 内置向量存储支持：原生支持 pgvector、Milvus、Pinecone 等向量数据库，简化 RAG 开发
4. 结构化输出：通过 `BeanOutputConverter` 将 LLM 输出直接映射为 Java 对象，无需手动解析 JSON

```
// 示例：Spring AI 结构化输出
var converter = new BeanOutputConverter<>(ResumeAnalysisDTO.class);
String result = chatClient.prompt()
    .system(systemPrompt)
    .user(userPrompt + converter.getFormat())
    .call()
    .content();
return converter.convert(result);  // 直接得到 Java 对象
```

数据存储为什么选择 PostgreSQL + pgvector?

本项目需要同时存储结构化数据（简历、面试记录）和向量数据（文档 Embedding）。方案对比：

方案	优点	缺点
PostgreSQL + pgvector	一套数据库搞定，运维简单	向量检索性能不如专业向量库
PostgreSQL + Milvus	向量检索性能更好	多一个组件，运维复杂度增加
PostgreSQL + Pinecone	云托管，无需运维	成本高，数据在第三方

选择 pgvector 的理由：

- 架构简单：不引入额外组件，降低部署和运维复杂度
- 性能够用：HNSW 索引支持毫秒级检索，万级文档场景完全够用
- 事务一致性：向量数据和业务数据在同一数据库，天然支持事务
- SQL 查询：可以结合 WHERE 条件过滤，比如"只在某个分类的知识库中检索"

```
-- pgvector 相似度搜索示例
SELECT content, 1 - (embedding <=> $1) as similarity
FROM vector_store
WHERE metadata->>'category' = 'Java'
ORDER BY embedding <=> $1
LIMIT 5;
```

为什么不选择 MySQL 搭配向量数据库呢?

PostgreSQL 最大的优势，也是它在 AI 时代甩开对手的“王牌”，就是其强大的可扩展性。开发者可以在不修改内核的情况下，像“即插即用”一样为数据库安装各种功能强大的插件，这让 PostgreSQL 变成了一个无所不能的“数据瑞士军刀”。

- **AI 向量检索?** 有官方推荐的 **pgvector** 扩展，性能强大，生态成熟，足以媲美专业的向量数据库。
- **全文搜索?** 内置支持（能满足基础需求），或使用 **pg_bm25** 等扩展。
- **时序数据?** 有顶级的 **TimescaleDB** 扩展。
- **地理信息?** 有行业标准的 **PostGIS** 扩展。

这种“一站式”解决能力，正是其魅力所在。它意味着许多项目不再需要依赖 Elasticsearch、Milvus 等大量外部中间件，仅凭一个增强版的 PostgreSQL 即可满足多样化需求，从而极大地简化了技术栈，降低了开发和运维的复杂度与成本。

关于 MySQL 和 PostgreSQL 的详细对比，可以参考我写的这篇文章：MySQL vs PostgreSQL，如何选择？。

为什么引入 Redis?

本项目主要有两个场景用到了 Redis：

1. Redis 替代 `ConcurrentHashMap` 实现会话的缓存。
2. 基于 Redis Stream 实现简历分析、知识库向量化等场景的异步（还能解耦，分析和向量化可以使用其他编程语言来做）。

为什么引入 Redis Stream？为何不选择 Kafka、RabbitMQ 等更成熟的消息队列?

简历分析、知识库向量化等 AI 任务耗时较长（10-60 秒），不适合同步处理。需要消息队列实现异步解耦。

维度	Redis Stream	RabbitMQ	Kafka	内存队列
吞吐量	高（十万级 QPS）	中（万级 QPS）	极高（百万级，水平扩展）	极高（千万级/秒，受限于 CPU/内存）
延迟	极低（亚毫秒级）	低（毫秒级）	中（毫秒到十毫秒级）	极低（纳秒/微秒级）
持久化	支持（RDB/AOF）	支持（Mnesia/磁盘）	强支持（原生分段日志）	无（进程终止即失）
消息堆积能力	一般（受限于内存）	中（磁盘堆积，性能下降明显）	极强（TB 级磁盘存储）	差（受限于堆内存）
消费模式	发布订阅 / 消费者组	灵活路由 / 多种交换机模式	发布订阅 / 消费者组	点对点 / 多消费者（取决于实现）
消息回溯	支持（按 ID / 时间范围）	不支持	强支持（按 Offset / 时间戳）	不支持
消息顺序性	单 Stream 有序	单队列有序	单 Partition 有序	有序（单队列）
可靠性	中（异步复制可能丢失）	高（Publisher Confirm / 事务）	极高（多副本 ISR + acks）	低（无持久化、无确认）
运维复杂度	低	中	高（KRaft 模式已简化）	极低
适用场景	轻量级流处理、已有 Redis 基础设施	复杂路由、企业级集成	大数据流、事件溯源、日志聚合	进程内解耦、极致性能场景

选择 Redis Stream 的理由：

- 复用现有组件：Redis 已用于会话缓存，无需引入新中间件
- 功能满足需求：支持消费者组、消息确认（ACK）、持久化
- 运维简单：对于中小型项目，Redis Stream 完全够用

构建工具为什么选择 Gradle?

SpringBoot 官方现在用的就是 Gradle，加上国内现在都是 Maven 更多，换个 Gradle 还更新颖一些。

个人也更喜欢用 Gradle，也写过相关的文章：Gradle 核心概念总结。

为什么使用 MapStruct?

项目中有大量 Entity ↔ DTO 转换需求，MapStruct 是编译时代码生成的对象映射框架：

方案	性能	类型安全	使用复杂度
MapStruct	零反射，最快	编译时检查	定义接口即可
BeanUtils	反射，慢	运行时报错	一行代码
ModelMapper	反射，较慢	运行时报错	配置复杂
手写转换	最快	编译时检查	重复代码多

为什么使用 Apache Tika?

系统需要解析多种格式的文档（PDF、Word、TXT），Apache Tika 是 Apache 基金会的文档解析库：

- 格式支持全：PDF、DOCX、DOC、TXT、HTML、Markdown 等上百种格式
- 自动识别：根据文件内容自动检测格式，无需依赖文件扩展名
- 文本提取：统一的 API 提取纯文本，屏蔽格式差异

```
// Tika 解析示例
Tika tika = new Tika();
String content = tika.parseToString(inputStream);  // 自动识别格式并提取文本
```

为什么使用 SSE 而不是 WebSocket?

知识库问答需要流式输出（像 ChatGPT 那样逐字显示），有两种技术选择：

方案	优点	缺点
SSE	简单，基于 HTTP，单向推送	仅支持服务端 → 客户端
WebSocket	双向通信，功能强大	协议复杂，需要维护连接状态

选择 SSE 的理由：

- 场景匹配：LLM 流式输出是单向的（服务端 → 客户端），不需要双向通信
- 实现简单：基于 HTTP，天然支持重连、跨域
- Spring 支持好：`Flux<ServerSentEvent<String>>` 一行代码搞定

前端为什么选择 React + TypeScript + Tailwind CSS?

技术	选择理由
React	生态最成熟，组件化开发，社区资源丰富
TypeScript	类型安全，IDE 智能提示，减少运行时错误
Vite	开发服务器启动快（秒级），HMR 热更新体验好
Tailwind CSS	原子化 CSS，快速开发，无需写 CSS 文件

效果展示

简历与面试

简历库：

简历上传分析：

简历分析详情：

社招简历1.pdf
上传于 2026/01/02

开始模拟面试

简历分析 | 面试记录 2

核心评价

导出分析报告

候选人具备扎实的Java后端开发能力，项目经历覆盖物联网、智慧社区等复杂业务场景，技术选型合理且有一定深度，但在表达精炼度、量化结果呈现和技术术语规范性方面仍有提升空间。

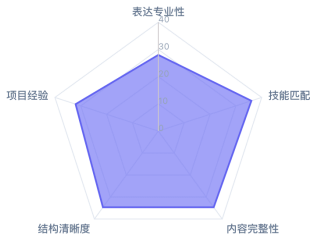
总分
78 / 100

分析时间
2026/01/02 12:12

优势亮点

- 具备多领域项目经验（IBMS、智慧社区、人脸识别），业务覆盖面广
- 熟练掌握主流技术栈，包括Spring Boot、Kafka、Redis、Elasticsearch等
- 在性能优化方面有实际动手能力，如CompletableFuture异步编排、SQL与索引优化
- 使用设计模式（策略、责任链）解决复杂告警逻辑，体现一定架构思维
- 有CI/CD流水线搭建和代码治理实践经验

多维度评分



项目经验	32/40
技能匹配	18/20
内容完整性	13/15
结构清晰度	13/15
表达专业性	7/10

改进建议 (9 条)

高优先级 (4)

高 项目
项目描述中缺乏统一的STAR结构，部分成果未明确量化
将'优化了一张图相关统计接口'改为：'主导辖区数据一张图统计接口优化，通过CompletableFuture并行聚合5类数据源，RT从1.8s降至400ms；结合Redis双写缓存+MyBatis二级缓存，热点数据查询命中率提升至92%'

高 项目
人脸检索性能优化虽提及响应时间，但缺少QPS、数据规模等上下文支撑
重写为：'重构人脸检索服务，基于Elasticsearch构建倒排索引，支持百万级抓拍记录毫秒检索；通过分页深度限制+ILM策略控制内存增长，P99延迟由20s降至2s，QPS达1200+'

高 项目
设备接入模块提到异步编程，但未说明具体实现方式（是否使用CompletableFuture或线程池）
明确技术实现：'采用CompletableFuture+自定义线程池实现设备批量导入异步化，单次导入耗时从45s缩短至8s，吞吐量提升5倍'

高 表达
频繁使用'负责...开发''工作内容'等被动表述，不符合高级工程师表达习惯
全部改为主动动词开头：'主导''设计''重构''优化''搭建'。例如将'开发了可扩展的第三方平台对接模块'改为'设计并实现基于模板方法模式的第三方平台对接框架'

中优先级 (4)

中 技能
技术栈描述存在不规范命名，如'spring 外部化配置'应为'Spring Externalized Configuration'
统一技术名词大小写：Spring Boot、MySQL、Redis、Kafka、Elasticsearch、MyBatis、Nginx、GitLab、Jenkins、Docker、MQTT、Oracle

中 内容
模块顺序混乱，个人信息重复出现两次
调整简历结构顺序为：个人信息 → 求职意向 → 教育经历 → 专业技能 → 工作经历 → 项目经历 → 证书奖项（可选）→ 校园经历（可选）→ 个人评价

中 项目
第三方平台对接模块未体现扩展性和复用价值的具体指标
补充量化成果：'封装6个通用Starter组件，支撑8个外部平台快速对接，平均接入周期从5人日缩短至1.5人日'

中 项目
Logstash同步SQL优化中指出MyISAM引擎问题，但未说明为何不迁移至InnoDB
补充后续动作：'推动核心表迁移至InnoDB存储引擎，解决表锁争用问题，并配合索引优化使同步延迟稳定在200ms以内'

低优先级 (1)

低 技能
'了解计算机组成原理'对Java岗位价值较低，建议替换为更相关知识
替换为：'理解JVM内存模型与GC机制，具备MAT分析OOM问题的初步经验'以增强技术纵深感

面试记录：

AI Interview
智能面试助手
简历与面试
上传简历
AI 分析简历
简历库
管理所有简历
面试记录
查看面试历史
知识库
知识库管理
管理知识文档
问答助手
基于知识库问答
面试记录
查看和管理所有模拟面试记录
搜索简历名称...
面试总数
3
已完成
2
平均分数
76 分
关联简历
题目数
状态
得分
创建时间
操作
京东科技-三年经验-物联平台和能碳管理系统.pdf
#c50644fc
6 题
已创建
-
2026/01/03
社招简历1.pdf
#3dff4b03
6 题
已完成
76
2026/01/02
京东科技-三年经验-物联平台和能碳管理系统.pdf
#5b7b4ecb
5 题
已完成
76
2025/12/30

面试详情：

面试详情 #ff4b03

完成于 2026/01/02

导出 PDF

76

总分

面试评估

候选人具备扎实的后端开发实战经验，在项目优化场景中有较强的工程思维和问题解决能力。尤其在并发编程（CompletableFuture应用）、数据库性能调优（MyISAM vs InnoDB对比）和缓存一致性设计方面展现出良好的技术判断力，能够结合业务特性做出合理的架构取舍。回答逻辑清晰，善于使用结构化表达，并能准确识别关键痛点如锁竞争、回表IO等。但在JVM底层机制和部分并发API细节上存在明显盲区，尤其是面对未准备的问题时缺乏应对策略，暴露出理论深度不足的问题。整体表现符合中级工程师水平，具备成长为高级工程师的潜力，但需加强系统性理论学习和技术广度拓展。

表现优势

- 具备出色的工程实践能力，能针对具体业务场景设计合理的并发与缓存方案
- 对数据库存储引擎底层差异有深刻理解，能从索引结构、锁机制、缓存策略多维度分析性能瓶颈
- 在缓存一致性设计中展现出良好的架构权衡能力，能根据更新频率和系统复杂度决定是否引入MQ
- 表达逻辑清晰，善于使用结构化方式（分点论述、对比表格）呈现技术观点
- 能够量化优化成果（如响应时间从800ms降至200ms），体现结果导向思维

改进建议

- 加强对JVM内存模型和垃圾回收机制的深入理解，建议阅读《深入理解Java虚拟机》并结合实际GC日志分析进行实践。
- 补充CompletableFuture在异常处理、任务编排方面的高级用法知识，掌握生产环境中异步编程的最佳实践。
- 系统学习Java并发编程中Future相关API的设计差异，特别是get()与join()的本质区别及其对线程中断的处理方式。
- 建议通过源码调试HashMap扩容过程，加深对rehash优化原理的理解，而不仅是记忆结论。
- 提升回答技术问题的完整性，避免出现'不知道'或'不清楚'的情况，应尝试基于已有知识合理推导并表达思考过程。

问答记录详情

1 项目经历 - 并发优化 得分: 95

在智慧社区平台项目中，你提到使用 CompletableFuture 优化了一张图统计接口的查询性能，请详细说明你是如何设计线程池的，以及为什么选择这种方式而不是并行流或其他并发方案?

你的回答

"针对这个面试问题，以下是为你准备的简化版高分回答：1. 线程池设计核心在设计线程池时，我遵循了资源隔离与参数精调的原则：隔离策略： 为“一张图”统计接口创建独立线程池，避免长耗时的统计任务阻塞核心业务（如登录、缴费）的公共线程池。参数配置： * 核心线程数： 采用 $CPU 核心数 \times 2$。因为统计接口涉及多个数据库/中间件查询，属于 IO 密集型任务。队列容量： 设置了 500 的有界队列，防止高并发下内存溢出（OOM）。拒绝策略： 使用 CallerRunsPolicy，当负载过高时让主线程回退执行，起到自然降载的作用。2. 方案对比：为什么选 CompletableFuture? 相比其他方案，CompletableFuture 在该场景下具有决定性优势：方案缺点为什么弃用ParallelStream (并行流)共享全局 ForkJoinPool容易由于一个接口耗时过长，导致整个系统的并行流全部卡死。CountDownLatch + Runnable编码复杂，无法链式处理异常处理困难，且多个查询任务之间若有依赖关系（如 A 查完才能查 B）很难写。CompletableFuture(胜出)异步非阻塞、支持任务编排（多表并行查，最后汇总）、可自定义线程池实现物理隔离。3. 落地效果性能提升： 将原本串行的 5 个 SQL/接口查询（总耗时约 800ms）改为并行执行，接口整体响应时间降低至耗时最长的那个子任务（约 200ms），性能提升了 4 倍。稳定性： 通过自定义线程池，确保了统计接口在大规模调用时不会影响到社区缴费、门禁等高频核心业务。"

AI 深度评价

回答非常出色，体现了对并发编程的深刻理解和工程实践经验。线程池参数设计合理，有明确的资源隔离意识；方案对比清晰，能从系统稳定性角度解释为何弃用并行流；性能提升数据量化具体，落地效果可信。唯一可改进点是未提及CompletableFuture的任务取消机制或超时控制，但这不影响整体高分。

参考答案

在智慧社区平台中，我为'一张图'统计接口设计了专用的ThreadPoolExecutor线程池：核心线程数设为CPU核心数×2（适用于IO密集型），最大线程数同核心线程数以保持稳定，使用SynchronousQueue或有界队列（如500容量）配合CallerRunsPolicy策略防抖。选择CompletableFuture而非并行流，是因为ForkJoinPool.commonPool()是JVM全局共享的，一旦某个长任务占用会导致所有依赖并行流的功能卡顿；而自定义线程池除了物理隔离外，还支持更精细的任务调度、监控和拒绝处理。相比于手动管理Thread+CountDownLatch的方式，CompletableFuture提供声明式的异步编排能力（thenCombine/applyToEither等），代码可读性和维护性更高。最终将原800ms串行查询优化至约200ms，QPS提升4倍以上，且不影响核心业务线程池稳定性。

2 MySQL - 索引与存储引擎 得分: 92

在XX人脸识别平台项目中，你发现Logstash同步SQL性能较差，虽然已走索引但耗时仍高。你提到表使用了MyISAM存储引擎和分区表，请解释MyISAM与InnoDB在索引结构和锁机制上的核心差异，以及这对查询和数据同步可能产生的影响。

模拟面试：

知识库

知识库管理：

问答助手：

项目结构

这是项目前后端的结构说明：

```
interview-guide/
├── app/                                  # 后端 Spring Boot 应用
│   ├── src/main/java/interview/guide/
│   │   ├── App.java                      # 启动类
│   │   |
│   │   ├── common/                       # 公共模块
│   │   |   ├── config/                   # 配置类（CORS、S3存储等）
│   │   |   ├── constant/                 # 常量定义
│   │   |   ├── exception/                # 全局异常处理
│   │   |   ├── model/                    # 公共模型（如异步任务状态）
│   │   |   └── result/                   # 统一响应封装 Result<T>
│   │   |
│   │   ├── infrastructure/               # 基础设施层
│   │   |   ├── file/                     # 文件处理（解析、存储、校验）
│   │   |   |   ├── DocumentParseService  # Apache Tika 文档解析
│   │   |   |   ├── FileStorageService    # S3 文件存储
│   │   |   |   ├── FileHashService       # SHA-256 哈希计算
│   │   |   |   └── FileValidationService # 文件类型/大小校验
│   │   |   ├── redis/                    # Redis 操作封装
│   │   |   |   ├── RedisService          # 通用 Redis 操作 + Stream 消费
│   │   |   |   └── InterviewSessionCache # 面试会话缓存
│   │   |   ├── mapper/                   # MapStruct 对象映射
│   │   |   └── export/                   # PDF 导出服务
│   │   |
│   │   └── modules/                      # 业务模块
│   |       ├── resume/                   # 简历模块
│   |       |   ├── ResumeController      # REST API
│   |       |   ├── model/                # Entity + DTO
│   |       |   ├── repository/           # JPA Repository
│   |       |   ├── service/              # 业务逻辑
│   |       |   |   ├── ResumeUploadService      # 上传处理
│   |       |   |   ├── ResumeGradingService     # AI 分析评分
│   |       |   |   └── ResumePersistenceService # 持久化
```

```
│   │       │   └── listener/              # 异步消费者
│   │       │       ├── AnalyzeStreamProducer   # 发送分析任务
│   │       │       └── AnalyzeStreamConsumer   # 消费并执行分析
│   │       │
│   │       ├── interview/                 # 面试模块
│   │       │   ├── InterviewController
│   │       │   ├── model/
│   │       │   ├── repository/
│   │       │   ├── service/
│   │       │   │   ├── InterviewSessionService   # 会话管理
│   │       │   │   ├── InterviewQuestionService  # 问题生成
│   │       │   │   └── AnswerEvaluationService   # 答案评估
│   │       │   └── listener/
│   │       │       └── EvaluateStreamConsumer    # 异步评估
│   │       │
│   │       └── knowledgebase/              # 知识库模块
│   │           ├── KnowledgeBaseController # 知识库管理 API
│   │           ├── RagChatController       # RAG 问答 API（支持 SSE）
│   │           ├── model/
│   │           ├── repository/
│   │           │   └── VectorRepository    # 向量数据操作
│   │           ├── service/
│   │           │   ├── KnowledgeBaseUploadService  # 上传处理
│   │           │   ├── KnowledgeBaseVectorService  # 向量化
│   │           │   ├── KnowledgeBaseQueryService   # RAG 检索 + 生成
│   │           │   └── RagChatSessionService       # 聊天会话管理
│   │           └── listener/
│   │               └── VectorizeStreamConsumer     # 异步向量化
│   │
│   └── src/main/resources/
│       ├── application.yml                 # 主配置文件
│       └── prompts/                        # AI 提示词模板
│           ├── resume-analysis-*.st        # 简历分析提示词
│           ├── interview-question-*.st     # 面试问题生成提示词
│           ├── interview-evaluation-*.st   # 面试评估提示词
│           └── knowledgebase-query-*.st    # RAG 问答提示词
│
├── frontend/                               # 前端 React 应用
│   ├── src/
│   │   ├── main.tsx                        # 入口文件
│   │   ├── App.tsx                         # 根组件 + 路由配置
│   │   │
│   │   ├── api/                            # API 请求层
│   │   │   ├── request.ts                  # Axios 封装 + 拦截器
│   │   │   ├── resume.ts                   # 简历 API
│   │   │   ├── interview.ts                # 面试 API
│   │   │   ├── knowledgebase.ts            # 知识库 API
│   │   │   └── ragChat.ts                  # RAG 聊天 API（含 SSE）
│   │   │
│   │   ├── pages/                          # 页面组件
│   │   │   ├── UploadPage.tsx              # 简历上传
│   │   │   ├── HistoryPage.tsx             # 简历列表
│   │   │   ├── ResumeDetailPage.tsx        # 简历详情 + 分析结果
│   │   │   ├── InterviewPage.tsx           # 模拟面试
│   │   │   ├── InterviewHistoryPage.tsx    # 面试记录
│   │   │   ├── KnowledgeBaseUploadPage.tsx # 知识库上传
│   │   │   ├── KnowledgeBaseManagePage.tsx # 知识库管理
│   │   │   └── KnowledgeBaseQueryPage.tsx  # 知识库问答
```

```
│   │   │
│   │   ├── components/                    # 通用组件
│   │   │   ├── Layout.tsx                 # 页面布局（侧边栏 + 内容区）
│   │   │   ├── FileUploadCard.tsx         # 文件上传卡片
│   │   │   ├── AnalysisPanel.tsx          # 分析结果展示
│   │   │   ├── InterviewChatPanel.tsx     # 面试问答交互
│   │   │   ├── RadarChart.tsx             # 雷达图（Recharts）
│   │   │   ├── CodeBlock.tsx              # 代码块高亮
│   │   │   └── ConfirmDialog.tsx          # 确认弹窗
│   │   │
│   │   ├── types/                         # TypeScript 类型定义
│   │   └── utils/                         # 工具函数
│   │
│   ├── package.json
│   └── vite.config.ts
│
├── docker/                                # Docker 相关
│   └── postgres/init.sql                  # 数据库初始化（pgvector 扩展）
│
├── docker-compose.yml                     # 一键部署编排
├── .env.example                           # 环境变量模板
└── README.md
```

后端分层详细介绍:

层级	目录	职责
Controller	`modules/*/Controller`	REST API 入口，参数校验，调用 Service
Service	`modules/*/service/`	业务逻辑，事务管理
Repository	`modules/*/repository/`	数据访问，JPA 查询
Model	`modules/*/model/`	Entity 实体 + DTO 传输对象
Listener	`modules/*/listener/`	Redis Stream 异步消费者
Infrastructure	`infrastructure/`	通用基础设施（文件、缓存、导出）
Common	`common/`	公共配置、异常、常量

学习本项目你将获得什么?

本项目采用最新主流的 Java 技术栈，是市面上第一个基于 SpringBoot4.x+SpringAI2.x 的实战项目以及教程。

项目整体难易程度一般，即使你的编程基础一般，按照项目教程也能顺利走完。

通过学习这个项目，你不仅能掌握最新的 Java AI 生态工具，还能学会如何构建一个生产级别的大模型应用。

深度掌握 AI 应用开发核心范式

本项目是学习 **LLM 应用开发**的绝佳实践案例:

- **Spring AI 2.0 实战**：掌握如何通过统一抽象接口对接多种大模型（如通义千问、OpenAI），实现“零成本”模型切换。

- **Prompt Engineering（提示词工程）**：学习如何编写结构化的 System/User Prompt，并利用 `BeanOutputConverter` 实现 **LLM 输出到 Java 对象的自动化映射**，告别繁琐的 JSON 手动解析。
- **RAG 全链路实现**：深度理解“文档解析 -> 文本分块 -> 向量化 (Embedding) -> 向量数据库存储 -> 相似度检索 -> 上下文增强生成”的完整闭环。

现代化的 Java 后端架构思维

本项目采用了目前 Java 社区最前沿的技术组合：

- **拥抱 Java 21 & Spring Boot 4.0**：提前布局最新版本的特性（如record、虚拟线程等），让你的技术栈领先市场。
- **模块化单体架构**：学习如何通过清晰的分层逻辑（Modules + Infrastructure + Common）组织代码，使系统既具备微服务的解耦特性，又保留单体应用的部署便捷性。
- **高性能对象转换**：通过 **MapStruct** 替代传统的反射拷贝，学习在追求极致性能的场景下如何优雅地处理 Entity 与 DTO 的映射。

高级数据存储与中间件应用

避开盲目引入复杂中间件的坑，学习“最合适”的选型逻辑：

- **PostgreSQL + pgvector 的“一站式”方案**：学习如何使用一套数据库同时处理关系型业务数据和 AI 向量数据，掌握 **HNSW 索引**在万级文档场景下的优化实践。
- **Redis Stream 异步任务处理**：深入理解为什么在 AI 耗时任务（10-60s）场景下选择 Redis Stream 而非 Kafka。掌握基于 **消息队列** 实现系统解耦和流量削峰的真实手段。
- **文件处理专家系统**：利用 **Apache Tika** 构建通用的文档解析引擎，处理 PDF、Word、TXT 等多种格式，提升处理非结构化数据的能力。

工程化与部署能力

- **标准化交付**：学习使用 **Gradle** 进行项目构建，掌握其相比 Maven 的灵活性优势。
- **容器化部署**：通过 **Docker Compose** 实现数据库（含扩展）、缓存、后端应用的一键式环境搭建，理解生产环境下的基础设施配置。

前端工程化与交互设计

对于后端同学，这也是一次绝佳的“全栈能力”升级机会：

- **SSE (Server-Sent Events) 流式渲染**：掌握像 ChatGPT 一样逐字输出回答的实现技术，理解其相比 WebSocket 在单向推送场景下的优势。
- **响应式与动画设计**：使用 **Tailwind CSS** 快速构建美观的 UI，结合 **Framer Motion** 提升用户交互体验。
- **数据可视化**：通过 **Recharts** 将 AI 分析后的简历评分、维度对比以直观的雷达图等形式展现。

总结

很多 AI 项目只停留在调用一个 API。而本项目带你解决的是**真实工程问题**：

- 如何处理大模型响应慢的问题？（**异步处理 + Redis Stream**）
- 如何让大模型输出格式固定的数据？（**结构化 Prompt + MapStruct**）
- 如何让大模型基于私有文档回答？（**RAG + pgvector**）